

1 Analiză și Soluție

1.1 Descrierea Problemei

Problema Cercețasi (Scouterii) solicită numărarea unor funcții cu proprietăți speciale. Pe baza codului furnizat, putem deduce că trebuie să calculăm o sumă de tipul:

$$\text{ANS} = \sum_{i=st}^{dr} \frac{S(n, i)}{(n-i)!}$$

unde $S(n, k)$ reprezintă numerele lui Stirling de speța a doua, care numără numărul de partiții ale unei mulțimi cu n elemente în exact k submulțimi nevide.

1.2 Observații Cheie

Numerele lui Stirling de speța a doua pot fi calculate eficient folosind formula explicită bazată pe principiul includerii-excluzării:

$$S(n, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} \cdot i^n$$

Această formulă se poate rescrie convenient ca:

$$S(n, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} \cdot i^n = \frac{1}{k!} \sum_{i=0}^k (-1)^i \binom{k}{i} \cdot (k-i)^n$$

1.3 Derivarea Soluției

Din formula de mai sus obținem:

$$\frac{S(n, k)}{(n-k)!} = \frac{1}{k!(n-k)!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} \cdot i^n$$

Folosind identitatea combinatorială $\binom{n}{k} = \frac{n!}{k!(n-k)!}$, expresia devine o sumă de termeni care pot fi precalculați eficient.

Codul calculează în faza de preprocesare:

- $fact[i] = i! \pmod{mod}$
- $ifact[i] = (i!)^{-1} \pmod{mod}$ (inversa modulară a factorialului)
- $inv[i] = i^{-1} \pmod{mod}$

1.4 Structura Algoritmului

1. Se citesc valorile n, r, st, dr și se ajustează conform formulei $n = n - r + 1$, $st = st - r + 1$, $dr = dr - r + 1$.
2. Se calculează tabloul prefix $pref[i]$ care memorează sume alternate ale inverselor factorialilor:

$$pref[i] = \sum_{j=0}^i (-1)^j \cdot \frac{1}{j!}$$

Aceasta se realizează eficient în timp liniar $O(dr)$.

3. Se iterează pentru fiecare i de la 0 la dr :

$$\begin{aligned} pp &= i^n \cdot \frac{1}{i!} \pmod{mod} \\ add &= \sum_{j=st}^{dr-i} (-1)^j \cdot \frac{1}{j!} \pmod{mod} \text{ (folosind } pref) \\ ans &= ans + pp \cdot add \pmod{mod} \end{aligned}$$

Complexitatea finală este $O(dr + \log mod)$ pentru calculul puterilor, unde $dr \leq 100000$.

1.5 Note de Implementare

- Toate calculele se efectuează modulo $mod = 1000000007$ (un număr prim).
- Inversa modulară se calculează folosind teorema lui Fermat: $a^{-1} \equiv a^{mod-2} \pmod{mod}$.
- Pentru eficiență, $inv[i]$ se calculează recursiv: $inv[i] = -inv[mod \% i] \cdot \frac{mod}{i}$.
- Verificarea $st - i - 1 \geq 0$ este esențială pentru a evita indici negativi la accesarea tabloului $pref$.

1.6 Corectitudine

Formula implementată este echivalentă matematic cu suma cerută. Demonstrăm că:

$$\begin{aligned}
\text{ANS} &= \sum_{i=st}^{dr} \frac{S(n, i)}{(n-i)!} \\
&= \sum_{i=st}^{dr} \frac{1}{i!(n-i)!} \sum_{j=0}^i (-1)^{i-j} \binom{i}{j} j^n \\
&= \sum_{i=st}^{dr} \sum_{j=st}^i (-1)^{i-j} \frac{j^n}{j!(i-j)!(n-i)!}
\end{aligned}$$

Schimbând ordinea sumării și grupând convenabil termenii obținem exact algoritmul implementat, unde tabloul *pref* permite calculul rapid al sumelor interioare în $O(1)$.

1.7 Codul Complet

```

#include <bits/stdc++.h>
#define ll long long
using namespace std;

const int NMAX = 100008;
const ll mod = 1000000007;

int n, r, st, dr;
ll fact[NMAX], ifact[NMAX], inv[NMAX];

void pre(int en) {
    fact[0] = fact[1] = 1LL;
    ifact[0] = ifact[1] = 1LL;
    inv[0] = inv[1] = 1LL;
    for (ll i = 2; i <= en; i++) {
        fact[i] = (fact[i-1] * i) % mod;
        inv[i] = (inv[mod % i] * (mod - mod / i)) % mod;
        ifact[i] = (ifact[i-1] * inv[i]) % mod;
    }
}

ll comb(int n, int k) {
    if (n < 0 || k < 0 || k > n) return 0;
    return ((fact[n] * ifact[k]) % mod) * ifact[n-k]) % mod;
}

ll pw(ll baza, ll expo) {
    ll ans = 1LL;
    while (expo) {
        if (expo & 1) ans = (ans * baza) % mod;
    }
}

```

```

        expo >>= 1;
        baza = (baza * baza) % mod;
    }
    return ans;
}

ll get_stir(ll n, ll k) {
    if (n == 0 && k == 0) return 1LL;
    if (n == k) return 1LL;
    if (k > n || n == 0 || k == 0) return 0;

    ll ans = 0;
    for (int i = 0; i <= k; i++) {
        ll add = (comb(k, i) * pw(i, n)) % mod;
        if ((k - i) % 2 == 0) ans = (ans + add) % mod;
        else ans = (ans - add + mod) % mod;
    }
    return (ans * ifact[k]) % mod;
}

ll pref[NMAX];

void solve(string file) {
    ifstream fin(file + ".in");
    ofstream fout(file + ".out");

    fin >> n >> r >> st >> dr;
    pre(n);

    n -= r;
    n++;
    st -= r;
    st++;
    dr -= r;
    dr++;

    pref[0] = 1LL;
    for (ll i = 1; i <= dr; i++) {
        if (i & 1) pref[i] = (pref[i-1] - ifact[i] + mod) %
            mod;
        else pref[i] = (pref[i-1] + ifact[i]) % mod;
    }

    ll ans = 0;
    for (ll i = 0; i <= dr; i++) {
        ll pp = (pw(i, n) * ifact[i]) % mod, add;
        if (st - i - 1 >= 0)
            add = (pref[dr-i] - pref[st-i-1] + mod) % mod;
        else add = pref[dr-i];
    }
}

```

```

        add = (add * pp) % mod;
        ans = (ans + add) % mod;
    }
    fout << ans << "\n";
    fin.close();
    fout.close();
}

int main() {
    solve("cercetasi");
    return 0;
}

```